Ders 11 Genişletilmiş Notu: Quicksort Analizi, Arama Yöntemleri ve Hashing'e Giriş

Ders 11 Genişletilmiş Notu: Quicksort Analizi, Arama Yöntemleri ve Hashing’e Giriş

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Geçen Haftadan: Hoare Bölmesi ve Quicksort
- 2. Quicksort’ta En İyi Durum
- 3. Quicksort’ta En Kötü Durum
- 4. Quicksort’ta Ortalama Durum
- 5. Pivot Seçimi
- 6. Arama Yöntemlerinin Karmaşıklığı
- 7. Hashing’e Giriş
- 8. Hash Fonksiyonu ve Hash Tablosu
- 9. İyi Bir Hash Fonksiyonunun Özellikleri
- 10. Temel Hash Fonksiyonları: Bölme ve Çarpma Yöntemi
- 11. Çakışma (Collision)
- Örnek Sorular

1. Geçen Haftadan: Hoare Bölmesi ve Quicksort

Ders, böl ve yönet (divide and conquer) konusunda kalınan yerden, Quicksort’tan devam etti. Bu hafta Quicksort’un analizi bitirilip hashing (hashing) konusuna geçildi. Hashing bu hafta ve gelecek hafta işlenecek.

Hoca önce geçen haftanın ana noktasını hatırlattı. Quicksort’ta temel kullanılan bölme algoritması Hoare bölmesidir (Hoare partition). Quicksort’ta emek bölme işlemine harcanır. Parçaları birleştirmek için ayrıca bir emek gerekmez. Mergesort’ta ise durum tam tersidir: bölme kolaydır, emek birleştirmede (merge) harcanır.

Hoca, geçen hafta görülen bir bölme izlemesini de sözlü olarak hatırlattı. Soldan sağa ve sağdan sola ilerlenir. Soldaki eleman pivottan büyük, sağdaki eleman pivottan küçükse ikisi yer değiştirir. Bu iş indisler buluşana kadar sürer ve en iyi durumda durma noktası dizinin tam ortası olur.

2. Quicksort’ta En İyi Durum

Hoca derse bir soruyla başladı: Quicksort’ta en iyi durum (best case) hangi şartta oluşur? Bir öğrenci “dizi zaten sıralıysa tek döngüyle kontrol edip en iyi durumu elde ederiz” dedi. Hoca bunun doğru olmadığını söyledi. Bir başka öğrenci doğru cevabı verdi: pivot her seferinde ortada, yani ortanca (median) değerde olursa en iyi durum oluşur.

> **Hocanın vurgusu:** Hoca bunu “kural” olarak ekrana yazdı ve “burası önemli” dedi: Pivotun orta noktadaki bir değeri temsil etmesi en iyi durumu oluşturur.

Pivot ortanca olunca dizi tam ikiye bölünür. İki yarı ayrı ayrı çözülür. Buna bölme ve birleştirme işlerinin maliyeti olan $f(n)$ eklenir. Quicksort'ta bu maliyet bölme işleminin karşılaştırma sayısıdır ve en iyi durumda n'dir. Hoca bunu şöyle açıkladı: indisler soldan ve sağdan ilerler ve tam ortada buluşur. Böylece bölme toplam n karşılaştırma yapar.

$$C_{best}(n) = 2\,C_{best}(n/2) + n, \quad n > 1, \qquad C_{best}(1) = 0$$

Master teoremi ile çözüm. Rekürans $T(n) = a\,T(n/b) + f(n)$, $f(n) \in \Theta(n^d)$ biçimindedir.

- $a = 2$ (iki alt problem)
- $b = 2$ (her alt problem yarı boyutta)
- $d = 1$ ($f(n) = n$)

$a = b^d$, yani $2 = 2^1$ olduğu için sonuç $\Theta(n^d \log n)$ olur:

$$C_{best}(n) \in \Theta(n \log_2 n)$$

Geriye doğru yerine koyma (backward substitution) ile çözüm. Hoca, Master teoremi kullanılmasaydı $n = 2^k$ alınarak geriye doğru yerine koyma yapılabileceğini söyledi. Bu çözüm de $n \log_2 n$ verir. Ara adımlar (ara adım eklendi):

$$C(2^k) = 2C(2^{k-1}) + 2^k = 2^2 C(2^{k-2}) + 2 \cdot 2^k = \cdots = 2^k C(1) + k \cdot 2^k = n \log_2 n$$

Python ile $n = 2, 4, \ldots, 1024$ için rekürans hesaplandı. Her durumda sonuç tam olarak $n \log_2 n$ çıktı.

2.1 Ödevle Bağlantı: En Yakın Nokta Çifti

Hoca burada ödevin birinci sorusunu hatırlattı. Kaba kuvvet (brute force) olmayan, verimli çözümde en küçük uzaklığı (minimum distance) bulmanın karmaşıklığı nedir?

Soru: Verimli çözümün karmaşıklığı nedir, açık hali nedir? **Cevap:** Bir öğrenci önce $n \log n$, sonra açık halini $n \log n + n$ olarak verdi. Önce sıralama (sort) yapılır, sonra elemanlar sırayla karşılaştırılır. Hoca bunun bir ön sıralama (presorting) yaklaşımı olduğunu söyledi. $n \log n$ daha büyük olduğu için n terimi düşer ve karmaşıklık $O(n \log n)$ kalır.

3. Quicksort'ta En Kötü Durum

Hoca örnek dizi olarak $1, 2, 3, 4, 5, 6, 7$'yi verdi ve pivot 1 iken en kötü durumu (worst case) analiz etmeyi istedi.

Soru: En kötü durum dizinin tersten sıralı olması mı, yoksa sıralı olması mı? **Cevap:** Bir öğrenci "dizinin sıralı olması" dedi. Hoca bu cevapla devam etti.

3.1 Önce Yanlış Bir Senaryo

Bir öğrenci önce şöyle bir senaryo anlattı: pivot 1 iken 2 ile 7, sonra 3 ile 6, sonra 4 ile 5 karşılaştırılır. Hiçbiri yer değiştirmez. Sonra 2'ye geçilir ve aynı işlem tekrar edilir. Her turda $n + 1$ karşılaştırma olur.

Hoca ekranda soldan sağa tarama (left-to-right scan) ve sağdan sola tarama (right-to-left scan) kısımlarını düzeltip yeniden sordu: Bu senaryo doğru mu? Durma noktaları gerçekten 4 ile 5'e mi gidecek?

3.2 Doğru Analiz

Bir başka öğrenci doğru cevabı verdi ve hoca bunu ayrıntılandırdı:

- **Soldan sağa tarama:** Pivottan (1'den) büyük ya da eşit ilk elemanı arar. Hemen $A[1] = 2$'de durur. İş bitmiştir, daha ileri gitmez.

- **Sağdan sola tarama:** Pivottan küçük ya da eşit bir eleman arar. Kendinden küçük hiçbir eleman olmadığı için en başa, $A[0]$'a (pivotun kendisine) kadar gelir ve orada durur.
- i yukarıda, j aşağıda kalır. İndisler orta noktada buluşmaz. j en başa kadar gider.

Sonuçta bölme en uçta olur: pivotun solunda 0 eleman, sağında $n-1$ eleman kalır. Hoca, öğrencinin ilk anlattığı "4 ile 5'e doğru ilerleme" senaryosuna kimseden itiraz gelmediğini, bu yüzden doğrusunu iki kez vurguladığını söyledi.

3.3 Karşılaştırma Sayısı

n elemanlı ilk turda $n+1$ karşılaştırma yapılır. Bir sonraki turda n karşılaştırma yapılır ve bu böyle azalarak devam eder.

Soru: Bölerek en küçük hale indiğimizde, en son turda kaç karşılaştırma yapılır? **Cevap:** Bir öğrenci önce "7" dedi. Hoca cevabı verdi: En son turda 2 eleman kaldığında karşılaştırma sayısı 3'tür.

Bir öğrenci 3'ü anlamadığını söyleyince hoca 1 ve 2 kaldığı durumu açıkladı. Soldan ve sağdan gelen ilk karşılaştırmalar birer karşılaştırmadır: 2'nin pivottan küçük olup olmadığına, 1'in 2'den büyük olup olmadığına bakılır. Ardından i ile j'nin birbirini geçmesi gerekir: ya i sağa geçer ya da j azalarak sola geçer. Bu da üçüncü karşılaştırmadır. Hoca, aynı mantıkla n eleman varken de $n+1$ karşılaştırma yapıldığını hatırlattı.

> **Ders sonrası not:** Levitin'in Hoare bölmesinde sayılan karşılaştırmalar anahtar karşılaştırmalarıdır. 2 elemanlı $(1,2)$ son turda sol tarama 1 karşılaştırmada ($A[1] = 2 \geq 1$) durur. Sağ tarama ise önce $A[1] = 2$'ye, sonra $A[0] = 1$'e bakar ve 2 karşılaştırma yapar. Yani üçüncü karşılaştırma, j'nin i'yi geçerek pivotun kendisiyle karşılaştırılmasıdır. Toplamın 3 olduğu doğrudur. Python simülasyonu bu sayımı doğruladı.

Soru: Bu $f(n)$ fonksiyonunun kısaltılmış hali var mıdır? **Cevap:** Bir öğrenci $\frac{(n+1)(n+2)}{2} - 3$ dedi. Hoca bunu onayladı.

$$C_{worst}(n) = (n+1) + n + \cdots + 3 = \frac{(n+1)(n+2)}{2} - 3 \in \Theta(n^2)$$

Ara adım (ara adım eklendi): $1 + 2 + \cdots + (n+1) = \frac{(n+1)(n+2)}{2}$'dir. Bu toplamdan eksik olan $1 + 2 = 3$ çıkarılır.

$n = 7$ için: $8 + 7 + 6 + 5 + 4 + 3 = 33$ ve $\frac{8 \cdot 9}{2} - 3 = 33$. Python ile Hoare bölmesi $1, \ldots, 7$ dizisi üzerinde çalıştırıldı. Turlar sırasıyla 8, 7, 6, 5, 4, 3 karşılaştırma yaptı ve her turda sol parça boş kaldı. Toplam 33'tür.

3.4 Sonuç: Ne Zaman Mergesort?

Quicksort en kötü durumda, yani dizi sıralı olduğunda karesel (quadratic) karmaşıklığa sahiptir. Hoca bu yüzden elimizde bu tür bir veri grubu varsa Quicksort yerine Mergesort'un tercih edilebileceğini söyledi. Mergesort ayrıca kararlıdır (stable). Bunun şartı, bellek alanı (space) ile ilgili bir kısıtımızın olmamasıdır.

> **Ders sonrası not:** Derste bir öğrenci "pivotu en küçük değil en büyük değer olarak seçseydik de aynı sonuca ulaşmaz mıydık?" diye sordu. Hoca önce en küçük değer üzerinden analizi tamamlamayı önerdi, sonra bu soruya dönülmedi. Cevap evettir. Pivot ilk eleman seçildiğinde kesin azalan (ters sıralı) dizide de pivot her seferinde en büyük ya da en küçük eleman olur ve bölme en uçta gerçekleşir. Python simülasyonunda $7, 6, \ldots, 1$ dizisi de toplam 33 karşılaştırma verdi. Bu durumda da karmaşıklık $\Theta(n^2)$'dir.

4. Quicksort'ta Ortalama Durum

Hoca ortalama durumu (average case) çok ayrıntılı analiz etmeyeceklerini, ama yaklaşımı göstermek istediğini söyledi.

Varsayım (assumption): Elimizde rastgele (random) sıralanmış bir dizi var. Bölme (partition) herhangi bir noktada oluşabilir. Bölme noktası s, $0 \leq s \leq n-1$ aralığındaki her konumda eşit olasılıkla, yani $1/n$ olasılıkla oluşur.

Buradan rekürans bağıntısı yazılır:

$$C_{avg}(n) = \frac{1}{n} \sum_{s=0}^{n-1} \left[(n+1) + C_{avg}(s) + C_{avg}(n-1-s) \right], \quad n > 1$$

$$C_{avg}(0) = 0, \qquad C_{avg}(1) = 0$$

Hoca formülün parçalarını şöyle açıkladı:

- $\frac{1}{n}$: Her elemanın pivot olma ihtimalini hesaba katar. Her bölme noktası eşit olasılıklıdır.
- $n+1$: Bölme işleminin maliyetidir, yani $f(n)$ fonksiyonunu temsil eder.
- $C_{avg}(s)$ ve $C_{avg}(n-1-s)$: Pivotun bir tarafında kalan ve diğer tarafında kalan parçaların maliyetidir. Hoca iki tarafın da yazılması gerektiğini vurguladı.
- Başlangıç koşulları (initial condition) 0 ve 1 için 0'dır.

Bu rekürans çözüldüğünde:

$$C_{avg}(n) \approx 2n \ln n \approx 1.39\, n \log_2 n$$

Yani ortalama durumda en iyi duruma göre yaklaşık %39 daha fazla karşılaştırma yapılır.

> **Ders sonrası not:** Hoca "$1.38\, n \log_2 n$, yaklaşık %38 daha fazla" dedi. $2 \ln n = 2 \ln 2 \cdot \log_2 n$ ve $2 \ln 2 \approx 1.386$ olduğu için değer yuvarlanınca 1.39 çıkar. Levitin'de de "$1.39\, n \log_2 n$" ve "yaklaşık %39" olarak geçer. Python ile rekürans $n = 10^6$'ya kadar sayısal olarak çözüldü. $C_{avg}(n)/(n \log_2 n)$ oranı $n = 100$ için 1.17, $n = 10^6$ için 1.31'dir. Alt dereceli terimler yüzünden oran 1.386'ya yavaş yaklaşır. Bu yüzden yaklaşım asimptotiktir.

5. Pivot Seçimi

Hoca Quicksort bölümünü kapatmadan önce bir soru daha sordu.

Soru: Quicksort'un performansını doğrudan etkileyen etkenlerden biri nedir? **Cevap:** Bir öğrenci "pivot seçimi" dedi.

Soru: Pivot seçimi nasıl yapılabilir? **Cevap:** Öğrenci dört yol saydı:

- En baştaki eleman seçilebilir.
- En sondaki eleman seçilebilir.
- Ortanca (median) seçilebilir.
- Rastgele (random) seçilebilir.

Hoca, böl ve yönet kısmını kapatırken azalt ve yönet (decrease and conquer) konusunda anlatılanların da unutulmamasını istedi.

> **Ek bilgi (derste anlatılmadı):** Rastgele pivot seçiminde en kötü durum hâlâ $\Theta(n^2)$ olabilir. Ama sıralı girdi gibi belirli bir girdi artık her zaman kötü sonuç vermez. Beklenen karşılaştırma sayısı her girdi için $\Theta(n \log n)$ olur. Uygulamada sık kullanılan bir yol da ilk, orta ve son elemanın ortancasını pivot almaktır (median-of-three).

6. Arama Yöntemlerinin Karmaşıklığı

Hoca ardından arama işlemlerine (search operations) geçti ve bir dizi üzerinde arama yaparken elimizde hangi yöntemlerin olduğunu sordu. Bir öğrenci doğrusal arama (linear search) ve ikili arama (binary search) dedi.

6.1 Doğrusal Arama

Doğrusal arama, diğer adıyla sıralı arama (sequential search), iki tür dizide uygulanabilir: sıralı olmayan veri kümesi (unsorted data set) ve sıralı veri kümesi (sorted data set).

Soru: Doğrusal aramanın en kötü durum karmaşıklığı nedir? **Cevap:** Bir öğrenci n dedi. Hoca dizinin sıralı olup olmamasının fark edip etmediğini sordu. Öğrenci "fark etmez" dedi. Hoca "emin misin?" diye sordu, öğrenci "evet" dedi ve hoca devam etti. En kötü durum $O(n)$'dir.

6.2 İkili Arama

> **Hocanın vurgusu:** İkili arama yapabilmek için veri kümesi mutlaka sıralı (sorted) olmalıdır.

Soru: İkili aramanın en kötü durum karmaşıklığı nedir? **Cevap:** Bir öğrenci $\log n$ dedi. En kötü durum $O(\log n)$'dir.

Python ile 1000 elemanlı sıralı dizide bütün değerler için arama yapıldı. En kötü durumda 10 karşılaştırma yapıldı. Bu, $\lfloor \log_2 1000 \rfloor + 1 = 10$ ile uyumludur.

6.3 Heap ile Arama

Hoca ikili arama dışında kullanılabilecek bir yapı sordu. Bir öğrenci heap dedi. Hoca bunun olabileceğini ama daha uygun bir arama yöntemi sorduğunu söyledi. Heap'te en büyük (max-heap) ya da en küçük (min-heap) eleman bellidir. Aşağı inerek bir elemanı bulabilirsiniz ama orada bazı sıkıntılar vardır.

> **Ek bilgi (derste anlatılmadı):** Heap yalnız ebeveyn ile çocuk arasındaki sırayı garanti eder. Kardeş düğümler ya da sol ve sağ alt ağaçlar arasında bir sıra yoktur. Bu yüzden heap'te herhangi bir anahtarı aramak en kötü durumda bütün düğümlere bakmayı, yani $O(n)$'i gerektirebilir.

6.4 İkili Arama Ağacı

Hocanın aradığı cevap ikili arama ağacıydı (binary search tree, BST).

Soru: İkili arama ağacı nasıl çalışıyordu? **Cevap:** Bir öğrenci şöyle anlattı: Düğümlerde $2i$ ve $2i + 1$ şeklinde ilerleniyordu. Büyük ya da küçük olma durumuna göre sola ya da sağa dallanılıyordu. Sol tarafta küçük, sağ tarafta büyük sayı vardır. Üstteki düğümün indisi 1 kabul edilirse alttakiler 2 ve 3 olur. İki katı olan indise küçük tarafa doğru, iki katı artı 1 olan indise büyük tarafa doğru dallanılır.

> **Ders sonrası not:** $2i$ ve $2i+1$ indisleri tam ikili ağacın (örneğin heap'in) dizi gösterimidir. İkili arama ağacı genellikle düğümler ve işaretçilerle (pointer) tutulur. BST'yi tanımlayan şey bu indisler değil, "sol alt ağaçtaki anahtarlar küçük, sağ alt ağaçtaki anahtarlar büyük" sıralama özelliğidir. Hoca öğrencinin cevabını düzeltmeden devam etti.

Soru: İkili arama ağacında aramanın en kötü durum karmaşıklığı nedir? **Cevap:** Bir öğrenci $\log n$ dedi. Başka bir öğrenci n dedi. Hoca neden n olduğunu sordu. Öğrenci şöyle açıkladı: Dizinin elemanlarını küçülen sırayla eklersek, örneğin 7'den sonra 6'yı, 6'dan sonra 5'i eklersek ağaç bu şekilde uzar. Hoca "aynen o şekilde" dedi.

Ağaç dengeli (balanced) değilse, eklenen elemanların sırasına bağlı olarak ağaç bir tarafa "yıkılır". Hoca örneği küçükten büyüğe ekleme üzerinden de verdi. Bu durumda ağaç dengeli olmaz ve en kötü durumda arama $O(n)$ olur. Bu, ikili arama ağacının bir dezavantajıdır.

Python ile $7, 6, 5, 4, 3, 2, 1$ sırasıyla eklenen bir BST kuruldu. Ağacın yüksekliği 7 çıktı, yani ağaç tek bir dala dönüştü.

Arama yöntemlerinin en kötü durum karmaşıklıkları:

Yöntem	Şart	En kötü durum
Doğrusal arama	Sıralı ya da sırasız dizi	$O(n)$

Yöntem	Şart	En kötü durum
İkili arama	Sıralı dizi	$O(\log n)$
İkili arama ağacı	Dengeli	$O(\log n)$
İkili arama ağacı	Dengesiz	$O(n)$

Hoca bunlara ek olarak hash yönteminin geleceğini söyledi.

7. Hashing'e Giriş

Soru: Hashing'de ne yapıyoruz? **Cevap:** Bir öğrenci "farklı veri setlerini bir fonksiyondan geçirip onları haritalama işlemi yapıyoruz" dedi.

Hoca bunu şöyle tamamladı: Elimizde bir anahtar (key) değeri vardır. Bu anahtarı bir hash fonksiyonuna (hash function) veririz. Hash fonksiyonu bize başka bir değer üretir. Üretilen bu değerin gösterdiği bölgeye gidip saklamak istediğimiz değeri (value) yazarız.

7.1 Doğrudan Adresleme Tablosu

Soru: Bunun, sınırsız sayıda anahtar kullanabildiğimiz, hashing olmadan yapılan versiyonuna ne ad verilir? **Cevap:** Bir öğrenci "arama tablosu (lookup table)" dedi. Hoca "tam olarak değil" dedi. Başka bir öğrenci "dümdüz indisleme" diye düşündüğünü söyledi. Hoca doğru terimi verdi: doğrudan adresleme tablosu (direct address table).

Hocaya göre bunu günlük hayatta zaten yapıyoruz: doğrudan bir anahtara erişip oraya değer atıyor, sonra yine oradan değere ulaşıyoruz. Doğrudan adresleme tablosu bildiğimiz dizilerdir. Uzaydaki bütün anahtarlar için birer alan kullanabilseydik, dizi mantığıyla $A[0]$, $A[1]$, $A[102]$, $A[1509]$, $A[12782]$ diyerek doğrudan erişip değeri atayabilir ya da oradan çekebilirdik.

7.2 Doğrudan Adresleme Tablosunun Sorunları

Soru: Doğrudan adresleme tablosunu neden kullanamıyoruz? **Cevap:** Öğrencilerden ve hocadan iki sorun çıktı.

Birinci sorun: Anahtar tam sayı olmayabilir. Bir öğrenci, indislerin tam sayı (integer) olduğunu, ama anahtar değerlerinin her zaman tam sayı olmadığını söyledi. Örneğin bir string değerini önce tam sayıya dönüştürmek gerekebilir. Hoca bunun bir sorun olduğunu kabul etti, ama string'i dönüştürebileceğimizi söyledi. Asıl soruyu şöyle sordu: Anahtar zaten sayısal bir değerse, her sayısal değer için hash kullanmadan doğrudan adresleme tablosunu mu kullanalım? Bu mümkün müdür?

İkinci sorun: Anahtar uzayı çok büyük. Hoca bir uzay düşünmemizi istedi. Uzayda önce 5 nokta vardır, sorun yoktur. Sonra 6, 7 olur ve uzay dolmaya başlar. Bir öğrenci örnek verdi: Uzayımız 0 ile 1.000.000 arasında olsun. Hash'te yalnız 5 sayı saklayacağız, ama doğrudan adreslemede bir milyon gözlük alan ayırmamız gerekir. Bunun yerine örneğin mod 5 kullanarak sayıları bir hash fonksiyonundan geçirir ve çok daha küçük bir dizide saklayabiliriz.

Hoca bu cevabı toparladı:

- Çok büyük miktarda elemanı saklamak için 2 milyon, 3 milyon, 5 milyonluk alan ayırmak gerekir. Bu maliyetli bir iştir.
- Ulaşmak istediğimiz değerler, örneğin string'den elde edilen değerler, çok büyük sayılar olabilir.
- Diziyi bir milyonluk açıp 991. adresi kullanırken 3. adresi hiç kullanmıyorsanız, bu "bonkör" bir kullanımdır ve israf anlamına gelir.

Kaynakları derli toplu kullanmak için, özellikle uzay yeterince büyüdüğünde, anahtarları karşı tarafta belirli adreslere eşlememiz (map) gerekir. Hoca bunu ekranda okla gösterdi: "bu buraya, bu buraya, bu da buraya map olsun". Bazı anahtarlar hash fonksiyonundan geçtikten sonra doğal olarak aynı adreslere düşecektir. Bu da hashing'in bir sorunudur: Uzaydaki farklı elemanların hash fonksiyonundan geçince aynı değeri üretmesi. Hoca bu sorunu çözmenin yollarının ilerleyen kısımda konuşulacağını söyledi.

7.3 Hashing'in Karmaşıklığı

Hashing'in karmaşıklığı $O(1)$'dir. Hoca, bazı durumlarda eleman ilk adımda bulunamasa da ve teoride $O(n)$'e gitme ihtimali olsa da, karşımıza hep $O(1)$ olarak çıkacağını söyledi. Nedeni ve ayrıntıları ilerleyen slaytlarda anlatılacak.

> **Ek bilgi (derste anlatılmadı):** Buradaki $O(1)$, iyi bir hash fonksiyonu ve makul bir doluluk oranı (load factor) varsayımı altındaki ortalama durumdur. Kötü bir hash fonksiyonu bütün anahtarları aynı adrese gönderirse arama, ekleme ve silme en kötü durumda $O(n)$ olur.

8. Hash Fonksiyonu ve Hash Tablosu

Hoca hashing'i biraz daha derinlemesine anlattı. Anahtar bir öğrenci numarası (student ID), TC kimlik numarası, personel numarası ya da herhangi bir benzersiz (unique) kimlik olabilir. Önemli olan, bir kişiyi ya da bir varlığı temsil etmesidir.

Anahtarı bir kara kutuya (black box) verirsiniz. Bu kara kutunun içinde aslında bir hash fonksiyonu vardır. Fonksiyon size bir adres değeri üretir. Bu adresi kullanarak bellek alanında o adrese gider ve ilgili değeri yerleştirirsiniz. Bu alana hash tablosu (hash table) denir.

Hash tabloları çok sık karşılaşılan bir veri yapısıdır. Hoca Java'dan da bilindiğini hatırlattı. Amaç anahtar-değer (key-value) çiftlerini saklamak ve özellikle arama işlemlerini daha hızlı hale getirmektir.

8.1 Hash Tablosunda Temel İşlemler

Soru: Hash tablosunda genelde hangi işlemler yapılır? **Cevap:** Hoca kendisi cevapladı: arama (search), ekleme (insert) ve silme (delete).

Bu işlemler sıklıkla yapılıyorsa hash tablosu tercih edilir. Hoca bunu diğer veri yapılarıyla karşılaştırdı:

- **Dizi:** Arama rahatlıkla yapılabilir, ama ekleme maliyetlidir.
- **Bağlı liste (linked list):** Ekleme daha az maliyetlidir, ama arama maliyetli hale gelir.
- **Hash tablosu:** Bu üç işlem sıklıkla ve hızlı yapılır.

Hocaya göre bu veri yapılarının hepsinin artı ve eksi yönleri vardır.

9. İyi Bir Hash Fonksiyonunun Özellikleri

Hoca bir hash fonksiyonu belirlerken dikkat edilecek üç özelliği saydı:

- **Kolay hesaplanabilirlik (easy to compute):** Hash fonksiyonunun kolay hesaplanabilmesi önemlidir. Fonksiyonun en az sayıda matematik işlemine sahip olması tercih edilir.
- **Tutarlılık (consistency):** Fonksiyon belirlenirken tutarlılığa dikkat edilir.
- **Düzgün adres üretimi (uniform address generation):** Hoca bunun "çok önemli" olduğunu söyledi.

  > **Ek bilgi (derste anlatılmadı):** Tutarlılık, aynı anahtarın her seferinde aynı adresi üretmesi demektir. Aksi halde bir anahtar eklendiği adreste sonra aranırken bulunamaz.

Soru: Düzgün adres üretimi neden önemlidir? **Cevap:** Bir öğrenci şöyle dedi: Fonksiyon her anahtar için ya da birkaç anahtar için aynı değeri üretirse, bu anahtarları aynı yere kaydetmemiz gerekir ve sorun oluşur. Hoca cevabı onayladı.

Hoca bunu mod alma işlemiyle örnekledi. Mod alma işlemi çok sık kullanılır. Mod alınan sayı genellikle tablonun boyutuyla doğrudan ilişkilidir. Bu sayıyı 10 seçerseniz 10, 40, 50 ve 100 anahtarlarının hepsi aynı değeri üretir:

$$10 \bmod 10 = 40 \bmod 10 = 50 \bmod 10 = 100 \bmod 10 = 0$$

Bu, düzgün bir adres üretme şekli değildir. Bu yüzden modüler işlemde ağırlıklı olarak asal sayılar (prime number) kullanılır. Mümkünse 10'un katı ve 2'nin katı olan sayılardan uzak durulur.

Python ile doğrulandı: Bu dört anahtarın mod 10 değeri dört kez 0'dır. Mod 11 alındığında ise sırasıyla 10, 7, 6, 1 değerleri çıkar ve hepsi farklı adreslere düşer.

10. Temel Hash Fonksiyonları: Bölme ve Çarpma Yöntemi

Soru: Bilinen en temel hash fonksiyonları nelerdir? **Cevap:** Hoca iki yaklaşım yazdı:

- Bölme yöntemiyle (division method) kullanılan hash fonksiyonları.
- Çarpma yöntemiyle (multiplication method) kullanılan hash fonksiyonları.

10.1 Bölme Yöntemi

Bölme yönteminde ağırlıklı olarak modüler aritmetik (modular arithmetic) kullanılır. Hoca, modüler aritmetiğin genelde tek yönlü (one-way) bir değer ürettiğini söyledi. Ona göre şifrelenen bir şeyi geri çözme (decrypt) şansı yoktur. Bu yüzden hashing kriptografik çözümlerde de sıklıkla kullanılır.

> **Ders sonrası not:** Hash tablolarında kullanılan k mod m gibi fonksiyonlar kriptografik değildir. $h(k) = k$ mod m değerinden aynı adrese düşen bir anahtar bulmak çok kolaydır, örneğin k, $k+m$, $k+2m$. Kriptografide SHA-256 gibi özel tasarlanmış, çakışmaya dayanıklı (collision-resistant) hash fonksiyonları kullanılır. Hocanın "tek yönlülük" vurgusu bu kriptografik hash fonksiyonları için geçerlidir.

Bölme yönteminde fonksiyon şöyledir:

$$h(k) = k \bmod m$$

Burada m genellikle tablonun uzunluğuna karşılık gelir.

Hocanın örneği: $m = 11$, $k = 25$.

- Verilen: tablo boyu $m = 11$, anahtar $k = 25$.
- İşlem: $25 = 2 \cdot 11 + 3$.
- Sonuç: $h(25) = 25 \bmod 11 = 3$.

Hash tablosunda 3 numaralı adrese gidilir ve 25 değeri oraya saklanır (store). Python ile $25 \bmod 11 = 3$ doğrulandı.

10.2 Çarpma Yöntemi

Çarpma yöntemi bu derste yalnız adıyla geçti. Ayrıntıları gelecek haftaya kaldı.

11. Çakışma (Collision)

Soru: Hashing'in, yani bir değeri fonksiyona verip yeni bir değer üretmenin dezavantajı nedir? **Cevap:** Hoca kendisi cevapladı: Birden fazla değerin aynı noktaya eşlenmesi (map). Buna çakışma (collision) denir.

Soru: Çakışma oluştuğunda bunu çözme şansımız var mı? **Cevap:** Bir öğrenci çözme şansımız olmadığını düşündüğünü söyledi. Hoca çözme şansımız olduğunu ve bunun yöntemleri bulunduğunu söyledi: açık hashing (open hashing), kapalı hashing (closed hashing), açık adresleme (open addressing) ve ayrık zincirleme (separate chaining) gibi yaklaşımlar. Bunlar sonraki derslerde görülecek.

Hoca çakışmayı kısaca bir kez daha tanımladı: Birden fazla eleman için aynı değerin üretilmesi durumunda, aynı adrese denk gelen iki elemanın nasıl saklanacağı problemi çakışma problemidir.

Ders hash fonksiyonları konusu tamamlanmadan bitti. Hoca hash fonksiyonlarının gelecek hafta bitirileceğini söyledi.

Örnek Sorular

Bu derste ekran görüntüsü ya da video kullanılmadı. Aşağıdaki sorular hocanın derste sözlü olarak sorduğu sorulardır.

Soru 1 (derste sözlü, 00:09:03): Quicksort'ta en iyi durum hangi şartta oluşur?

> **Çözüm (derste verildi):** Pivot her bölmede ortanca (median) değer olduğunda dizi tam ikiye bölünür. Bu en iyi durumdur. Dizinin önceden sıralı olması en iyi durum değildir.

Soru 2 (derste sözlü, 00:11:46): Quicksort'un en iyi durumunda bölme işleminin karşılaştırma sayısı kaçtır, neden? Rekürans nasıl çözülür?

> **Çözüm (derste verildi):** Bölme n karşılaştırma yapar, çünkü indisler tam ortada buluşur. $C_{best}(n) = 2C_{best}(n/2) + n$, $C_{best}(1) = 0$. Master teoreminde $a = 2$, $b = 2$, $d = 1$ olduğundan $C_{best}(n) \in \Theta(n \log_2 n)$ olur.

Soru 3 (derste sözlü, 00:15:38): Ödevin birinci sorusunda en küçük uzaklığı veren nokta çiftini bulan verimli çözümün karmaşıklığı nedir? Açık hali nedir?

> **Çözüm (derste verildi):** Önce sıralama, sonra sırayla karşılaştırma yapılır (ön sıralama). Açık hali $n \log n + n$'dir. Baskın terim kalır ve karmaşıklık $O(n \log n)$ olur.

Soru 4 (derste sözlü, 00:17:29): $1, 2, 3, 4, 5, 6, 7$ dizisinde pivot 1 iken Quicksort'un en kötü durumunu analiz edin. En kötü durum sıralı dizi mi, ters sıralı dizi mi?

> **Çözüm (derste verildi):** Sıralı dizide soldan sağa tarama $A[1] = 2$'de durur. Sağdan sola tarama $A[0]$'a kadar gider. Bölme en uçta olur. Turlar $n + 1, n, \dots, 3$ karşılaştırma yapar. $n = 7$ için $8 + 7 + 6 + 5 + 4 + 3 = 33$ karşılaştırma yapılır.

Soru 5 (derste sözlü, 00:23:28): Bölerek en küçük hale indiğimizde, en son turda kaç karşılaştırma yapılır?

> **Çözüm (derste verildi):** 2 eleman kaldığında 3 karşılaştırma yapılır. Sol tarama 1, sağ tarama 2 karşılaştırma yapar.

Soru 6 (derste sözlü, 00:26:01): En kötü durumdaki toplam karşılaştırma sayısının kısaltılmış hali nedir?

> **Çözüm (derste verildi):** $(n + 1) + n + \cdots + 3 = \frac{(n+1)(n+2)}{2} - 3 \in \Theta(n^2)$.

Soru 7 (derste sözlü, 00:20:11): Pivotu en küçük değil de en büyük değer olarak seçseydik de aynı sonuca ulaşmaz mıydık?

> **Çözüm (derste verilmedi):** Evet. Pivot ilk eleman seçildiğinde ters sıralı dizide $(7, 6, \dots 1)$ pivot her turda ya en büyük ya da en küçük eleman olur ve bölme hep en uçta gerçekleşir. Python simülasyonunda bu dizi için turlar yine 8, 7, 6, 5, 4, 3 karşılaştırma yaptı. Toplam 33'tür ve karmaşıklık $\Theta(n^2)$'dir.

Soru 8 (derste sözlü, 00:31:54): Quicksort'un performansını doğrudan etkileyen etken nedir? Bu seçim nasıl yapılabilir?

> **Çözüm (derste verildi):** Pivot seçimi. Pivot ilk eleman, son eleman, ortanca ya da rastgele seçilebilir.

Soru 9 (derste sözlü, 00:35:35): Doğrusal arama, ikili arama ve ikili arama ağacında aramanın en kötü durum karmaşıklığı nedir?

> **Çözüm (derste verildi):** Doğrusal arama $O(n)$'dir ve dizinin sıralı olması fark etmez. İkili arama $O(\log n)$'dir, ama dizi sıralı olmalıdır. İkili arama ağacı dengesizse, örneğin elemanlar sıralı ya da ters sıralı eklenirse, $O(n)$ olur.

Soru 10 (derste sözlü, 00:40:03): Doğrudan adresleme tablosunu neden kullanamıyoruz?

> **Çözüm (derste verildi):** Anahtar tam sayı olmayabilir ve önce dönüştürülmesi gerekir. Asıl sorun ise anahtar uzayının büyüklüğüdür. Örneğin 0 ile 1.000.000 arasındaki bir uzaydan yalnız 5 anahtar saklamak için bir milyon gözlük dizi açmak gerekir. Gözlerin çoğu boş kalır ve bu israftır.

Soru 11 (derste sözlü, 01:05:51): Hash tablosunda genelde hangi işlemler yapılır?

Çözüm (derste verildi): Arama (search), ekleme (insert) ve silme (delete).

Soru 12 (derste sözlü, 01:07:52): Düzgün adres üretimi neden önemlidir?

Çözüm (derste verildi): Fonksiyon farklı anahtarlar için aynı adresi üretirse bu anahtarlar aynı yere düşer ve sorun oluşur. Örneğin $m = 10$ ile 10, 40, 50 ve 100 anahtarlarının hepsi 0 adresine düşer. Bu yüzden m asal seçilir.

Soru 13 (derste sözlü, 01:11:19): Hashing'in dezavantajı nedir? Bu sorun çözülebilir mi?

Çözüm (derste verildi): Dezavantaj, birden fazla anahtarın aynı adrese düşmesidir (çakışma, collision). Çözülebilir. Açık adresleme (open addressing) ve ayrık zincirleme (separate chaining) gibi yöntemler vardır.

Dersin Toparlaması

Bu derste Quicksort'un analizi tamamlandı. Pivot her seferinde ortanca olduğunda dizi ikiye bölünür ve en iyi durum $\Theta(n \log n)$ olur. Pivot ilk eleman seçilip dizi sıralı verildiğinde bölme hep en uçta gerçekleşir. Bu durumda toplam karşılaştırma $\frac{(n+1)(n+2)}{2} - 3$, yani $\Theta(n^2)$ olur. Rastgele dizide ortalama durum yaklaşık $2n \ln n \approx 1.39\, n \log_2 n$'dir. Quicksort'un performansını belirleyen ana etken pivot seçimidir. Sıralı veriyle karşılaşılabilecekse ve bellek kısıtı yoksa kararlı olan Mergesort tercih edilebilir.

Ardından arama yöntemleri karşılaştırıldı: doğrusal arama $O(n)$, ikili arama sıralı dizide $O(\log n)$, ikili arama ağacı dengeliyse $O(\log n)$, dengesizse $O(n)$. Son olarak hashing'e giriş yapıldı. Doğrudan adresleme tablosunun büyük anahtar uzaylarında israfa yol açtığı, hash fonksiyonunun bu büyük uzayı küçük bir tabloya eşlediği görüldü. İyi bir hash fonksiyonu kolay hesaplanır, tutarlıdır ve adresleri düzgün dağıtır. Bölme yönteminde $h(k) = k \bmod m$ kullanılır ve m asal seçilir. Gelecek hafta çarpma yöntemi ve çakışma çözüm yöntemleriyle hash fonksiyonları konusu bitirilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.